

Faculty of Engineering, Ain
Shams University Spring
2026



[CSE211s] Introduction to Embedded Systems

Final Project Report

Team 12

Mechatronics Department SECTION 3 MCT

Name	ID	Section
Abdelrahman Hany Abdelrahman Omar	2100359	3 MCT
Khaled Amin Abdallah	2100808	3 MCT
Ziad shady mohie el dien	2100305	3 MCT

Submitted to:

Eng. Abdullah Awadallah

Table of Contents

Table of Contents	2
1. Objectives.....	3
2. Software Structure	3
2.1 Startup Code	3
2.2 main().....	4
2.3 Interrupt Service Routines (ISRs)	5
3. Concept Behind the Sensor	5
3.1 Physical Principle.....	5
4. Hardware Components	6
4.1 Pin-out Summary (NUCLEO-F401RE)	7
5. Project Deliverables	8
5.2 Source Code Repository	8
5.3 Drive Folder (Video)	8

1. Objectives

The objective of this project is to design and implement an embedded system, based on an STMicroelectronics Nucleo board, that uses an **HC-SR04 ultrasonic sensor** to measure the distance between the system and an obstacle placed in front of it. The system is mounted on a two-wheel differential-drive robot car so that it can react to that distance by moving the car forward, backward or stopping it. The detailed objectives are:

- Acquire range measurements from an HC-SR04 ultrasonic sensor in a non-blocking, interrupt-driven manner so that the main control loop is never delayed by I/O.
- Use the measured distance to follow an object that moves in a straight line in front of the car while keeping a suitable target distance from it.
- Stop the car safely whenever the followed object stops, leaves the sensor's field of view, or moves outside the configured maximum range.
- Drive two DC gear-motors through an L298N H-bridge using PWM speed control and digital direction control, exposing a clean forward / reverse / stop interface to the controller.
- Produce a modular, well-documented, and reusable codebase that clearly separates the sensor driver, the actuator driver, and the high-level control logic, as required by the course evaluation criteria.

2. Software Structure

File	Role
main.cpp	Startup banner, controller loop and telemetry.
ultrasonic.h / .cpp	Non-blocking, ISR-based HC-SR04 driver.
motor.h / .cpp	L298N differential-drive H-bridge driver.

2.1 Startup Code

The board boots from its on-chip ROM into the Mbed reset handler. The reset handler is supplied by the Mbed OS port for the target Nucleo board and performs the standard ARM Cortex-M startup sequence:

- Copy the initialised data section from Flash to SRAM.
- Zero the BSS section so that all uninitialised globals start at zero.
- Configure the system clock by calling `SystemInit()` (PLL setup, AHB/APB prescalers, Flash wait-states).
- Hand control to `_start()`, which calls the C++ runtime to construct global objects.

Two global objects are declared in main.cpp, so their constructors run before main():

```
static Ultrasonic sonar(TRIG_PIN, ECHO_PIN);
static Motor      drive(LEFT_PWM,  LEFT_DIR_A,  LEFT_DIR_B,
                        RIGHT_PWM, RIGHT_DIR_A, RIGHT_DIR_B);
```

The Ultrasonic constructor sets TRIG idle low, configures ECHO as an InterruptIn, and attaches the rising-edge and falling-edge callbacks to its own ISRs. The Motor constructor configures the two enable lines as PwmOut at a 20 kHz carrier (50 us period - inaudible and smooth) and the four direction lines as DigitalOut. After the global constructors return, _start() finally jumps to main().

2.2 main()

main() is the supervisor of the system. It is intentionally kept short and contains no blocking I/O - all communication with the hardware is delegated to the driver classes and to the interrupt service routines they install.

Once main() begins, it prints a startup banner over the virtual serial port, asks the motor driver to enter a safe stopped state, and starts the periodic ultrasonic sampling. From that point on it executes a deterministic loop with a 50 ms period (20 Hz):

- Read the latest measurement from the ultrasonic driver: the validity flag and the distance in centimetres.
- Compute the command. If the reading is invalid or the object is beyond MAX_RANGE the car is stopped; otherwise the controller maps the distance error (distance - target) to a signed duty cycle limited to the interval [-MAX_SPEED, +MAX_SPEED]. Inside a 3 cm dead-band the command is exactly zero, which prevents oscillation around the set point.
- Apply the command. A positive command drives the car forward, a negative one drives it backward, and zero stops both motors.
- Emit telemetry through printf() so the developer can monitor distance, validity and commanded duty cycle in real time.
- Sleep for the remaining slice of the 50 ms period using ThisThread::sleep_for(), releasing the CPU to the Mbed RTOS idle thread so the only events that wake the system between iterations are interrupts.

The behavioural parameters used inside main() are exposed as named constants at the top of the file so they can be tuned without touching the controller code:

```
TARGET_CM   = 20.0 cm    // suitable following distance
DEAD_BAND   =  3.0 cm    // tolerance around the target
MAX_RANGE    = 60.0 cm    // ignore objects past this
K_P          = 0.035      // proportional gain (1/cm)
MIN_SPEED    = 0.40       // duty needed to overcome motor stiction
MAX_SPEED    = 0.80       // upper duty limit
```

2.3 Interrupt Service Routines (ISRs)

The HC-SR04 driver uses four short ISRs. They keep the main loop free from any timing responsibility by reacting directly to the events generated by the sensor and by the Mbed timer subsystem. None of them performs blocking operations or floating-point calls beyond a single division - all updates to shared state are performed on volatile members so the latest value is visible to the main thread.

Source	ISR	Purpose
Ticker (60 ms)	Ultrasonic::_send_trigger	Emit a 10 us pulse on TRIG and arm the echo-timeout guard.
EXTI - ECHO rise	Ultrasonic::_on_echo_rise	Reset and start a high-resolution Timer; mark a measurement as in-flight.
EXTI - ECHO fall	Ultrasonic::_on_echo_fall	Stop the Timer, convert microseconds to centimetres, update the latest distance and validity flag.
Timeout (30 ms)	Ultrasonic::_on_echo_timeout	No echo within the maximum round-trip time - invalidate the reading so main() knows the target is out of range.

The Ticker ISR is the only one that issues a 10 us `wait_us()` inside an ISR; this is deliberate because the HC-SR04 datasheet requires a trigger pulse of at least 10 us, and busy-waiting that long inside an interrupt is acceptable. The two EXTI ISRs are essentially time-stamps - they read or write the Mbed Timer and set a volatile flag, nothing else. Because all heavy lifting (controller arithmetic, telemetry, PWM updates) happens in the main loop, the ISRs are short, deterministic, and free of priority-inversion hazards.

The motor driver does not need any interrupt of its own: the PWM is generated entirely in hardware by the STM32 advanced timers (TIM3_CH1 on D5 and TIM2_CH3 on D6) and the direction lines are simple DigitalOut writes.

3. Concept Behind the Sensor

The sensor used in this project is the HC-SR04 ultrasonic ranger. It is an active time-of-flight sensor: it emits a burst of ultrasound, listens for the echo reflected by the nearest obstacle, and reports the round-trip duration as a digital pulse whose width is proportional to the distance to the obstacle.

3.1 Physical Principle

Ultrasound is acoustic energy at frequencies above the human hearing limit (here, 40 kHz). The HC-SR04 contains two piezo-ceramic transducers - one acting as a transmitter and the other as a receiver. When a short electrical excitation is applied to the transmitter it vibrates and radiates an eight-cycle 40 kHz

acoustic burst into the air. The burst travels until it hits an obstacle, where part of its energy is reflected back. The reflected wave reaches the receiver, vibrates its membrane, and is converted back into an electrical signal that the on-board comparator transforms into a digital edge.

Because the speed of sound in air at room temperature is approximately 343 m/s, the wave covers 1 cm in roughly 29 us. The distance to the obstacle is therefore

$$\text{distance_cm} = \text{echo_pulse_width_us} / 58$$

where the factor $58 = 2 / (343 \text{ cm/us} / 10^4)$ accounts for the round-trip path (the sound travels to the obstacle and back).

4. Hardware Components

The following components were used to build the robot:

#	Component	Description and Role
1	STM32 Nucleo-F401RE board	Main microcontroller board (ARM Cortex-M4 at 84 MHz). Runs Mbed OS 6 and hosts the application firmware. Provides the GPIOs, the EXTI line for the echo capture, and the two independent advanced timers used to generate the motor PWMs (TIM3_CH1 on D5 and TIM2_CH3 on D6).
2	HC-SR04 ultrasonic sensor	40 kHz time-of-flight ranger with TRIG and ECHO pins, used to measure the distance between the car and the followed object.
3	L298N dual H-bridge motor driver	Drives the two DC gear-motors. Receives the PWM speed signals on ENA / ENB and the four direction bits on IN1..IN4. Its on-board 5 V regulator powers the logic side; the motor supply (VS) comes directly from the battery pack.
4	2 x DC gear-motors with wheels	Provide the propulsion. Each motor is independently controlled, enabling forward, reverse and (in principle) pivot turns through the differential-drive driver.
5	Robot chassis with front caster	Two-wheel differential-drive frame plus a free-running caster at the front for balance.
6	Battery pack	Provides power for the L298N motor supply. The Nucleo board can be powered through the same pack via the 5 V regulator on the L298N, or independently through its USB port during bench testing.
7	Voltage divider resistors (1 kOhm + 2 kOhm)	Step the 5 V ECHO signal down to a 3.3 V-safe level before it enters the Nucleo EXTI input.
8	Jumper wires & breadboard	Interconnect the Nucleo, sensor and motor driver according to the wiring shown below.

4.1 Pin-out Summary (NUCLEO-F401RE)

Function	Silkscreen	Notes
HC-SR04 TRIG	D8	Digital output.
HC-SR04 ECHO	D2	EXTI input, via 5 V to 3.3 V divider.
Left motor PWM	PWM/D5	TIM3_CH1 (independent timer).
Left motor IN1	D4	Direction bit (DigitalOut).
Left motor IN2	D7	Direction bit (DigitalOut).
Right motor PWM	PWM/D6	TIM2_CH3 (independent timer).
Right motor IN3	PWM/D9	Used as plain DigitalOut.
Right motor IN4	PWM/CS/D10	Used as plain DigitalOut.
GND	GND	Common ground (sensor, driver, Nucleo).
5 V (sensor)	5V	Powers the HC-SR04.

5. Project Deliverables

5.1 Source Code Repository

Full source code with comments (main.cpp, ultrasonic.h/.cpp, motor.h/.cpp):

Repository link: https://github.com/abeomar21/CSE211_T12FinalProject.git

5.2 Drive Folder (Video)

Drive folder link:

<https://drive.google.com/drive/folders/1Ez6z3bknP3SJmCjxKL9UYUShzbddKoWb?usp=sharing>